

Spring Boot Project Structure, Starters, DevTools, Actuator, and Configuration

In this lesson, we move beyond creating a basic Spring Boot application and explore the **tools, structure, and configuration that support real-world Spring Boot development**. We will examine the files and directories generated by Spring Initializr, understand how **Maven, Spring Boot Starters, and the Starter Parent** manage dependencies and project configuration, and introduce the basic architecture of **Spring MVC**. From there, we will improve the development workflow using **Spring Boot DevTools**, monitor applications with **Spring Boot Actuator**, explore basic endpoint security, and learn how to **package and run Spring Boot applications from the command line**. Finally, we will use *application.properties* to externalize custom values and configure important server settings such as the **port and context path**. The emphasis throughout the lesson is on understanding not only *how* to use these Spring Boot features, but also *why* they are useful in practical application development.

Learning Objectives

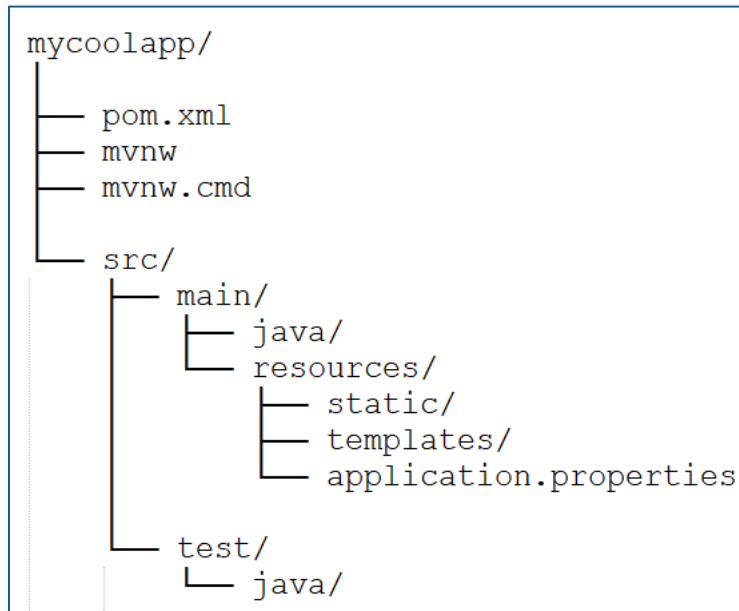
By the end of these notes, you should be able to:

- Navigate a Spring Boot Maven project.
- Explain the purpose of the Maven Wrapper and pom.xml.
- Explain how Spring Boot Starters simplify dependency management.
- Describe the role of the Spring Boot Starter Parent.
- Explain the basic Spring MVC request-processing architecture.
- Use Spring Boot DevTools for automatic application restart.
- Add and configure Spring Boot Actuator.
- Secure and selectively expose Actuator endpoints.
- Package and run Spring Boot applications from the command line.
- Define and inject custom application properties.
- Configure the embedded server using application.properties.

1. Exploring a Spring Boot Project

A project generated using **Spring Initializr** follows Maven's standard directory conventions. Understanding this structure is important because most Spring Boot projects organize source code, tests, configuration, and generated artifacts in predictable locations.

A simplified project looks like this:

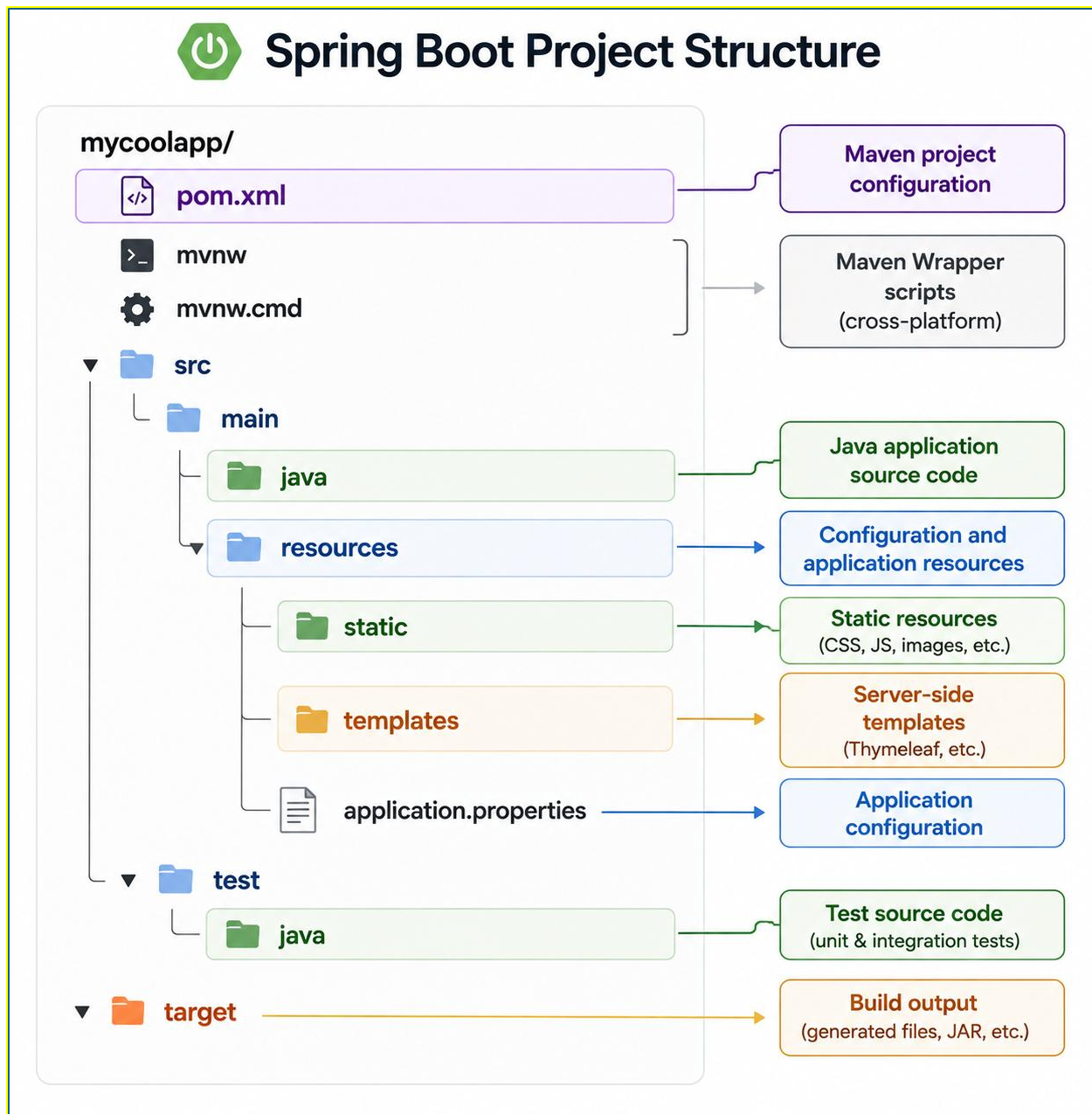


Key Directories

Location	Purpose
src/main/java	Java application source code
src/main/resources	Configuration and application resources
src/main/resources/static	Static resources
src/main/resources/templates	Server-side templates
src/test/java	Test source code
pom.xml	Maven project configuration



Spring Boot Project Structure



2. Maven Wrapper

Spring Initializr generates **Maven Wrapper** files that allow the project to use Maven without requiring a separately installed Maven executable on the system path. If the required Maven distribution is unavailable, the wrapper can obtain and use it.

The relevant launcher differs by operating system:

Windows: `mvnw.cmd`

Linux/macOS: `mvnw`

For example:

```
./mvnw clean
./mvnw compile
./mvnw test
./mvnw package
```

On Windows:

```
mvnw clean
mvnw compile
mvnw test
mvnw package
```

If Maven is installed directly, equivalent commands can be executed with:

```
mvn clean
mvn compile
mvn test
mvn package
```

Tip

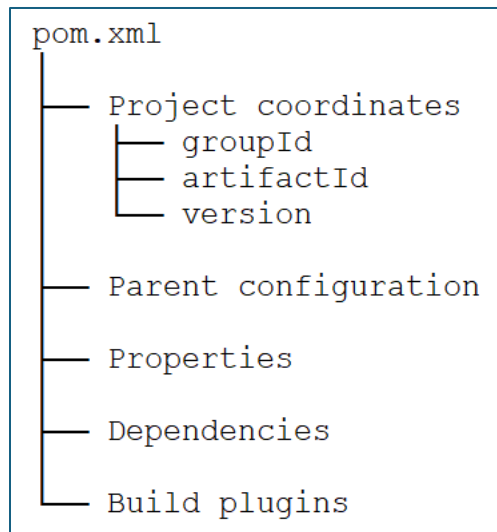
Using the Maven Wrapper helps different developers and build environments use the Maven setup associated with the project rather than depending entirely on a machine-wide Maven installation.

3. Understanding pom.xml

The Maven **Project Object Model**, or `pom.xml`, is the central Maven configuration file.

`Spring Initializr` generates it using the selections made when the project was created, including project coordinates and dependencies.

A Spring Boot POM typically contains information related to:



Two particularly important concepts are:

- **Spring Boot Starters**
- **Spring Boot Maven Plugin**

We examine each below.

4. Spring Boot Starters

4.1 The Dependency Problem

Building a Spring application may require several related libraries.

For example, a web application may require support for:

- Spring Web;
- Spring MVC;
- validation;
- JSON processing;
- an embedded web server.

Manually identifying compatible dependencies and versions can become cumbersome.

Spring Boot addresses this using **Starters**.

4.2 What Is a Spring Boot Starter?

A **Spring Boot Starter** is a convenient dependency descriptor that brings together a curated set of related dependencies.

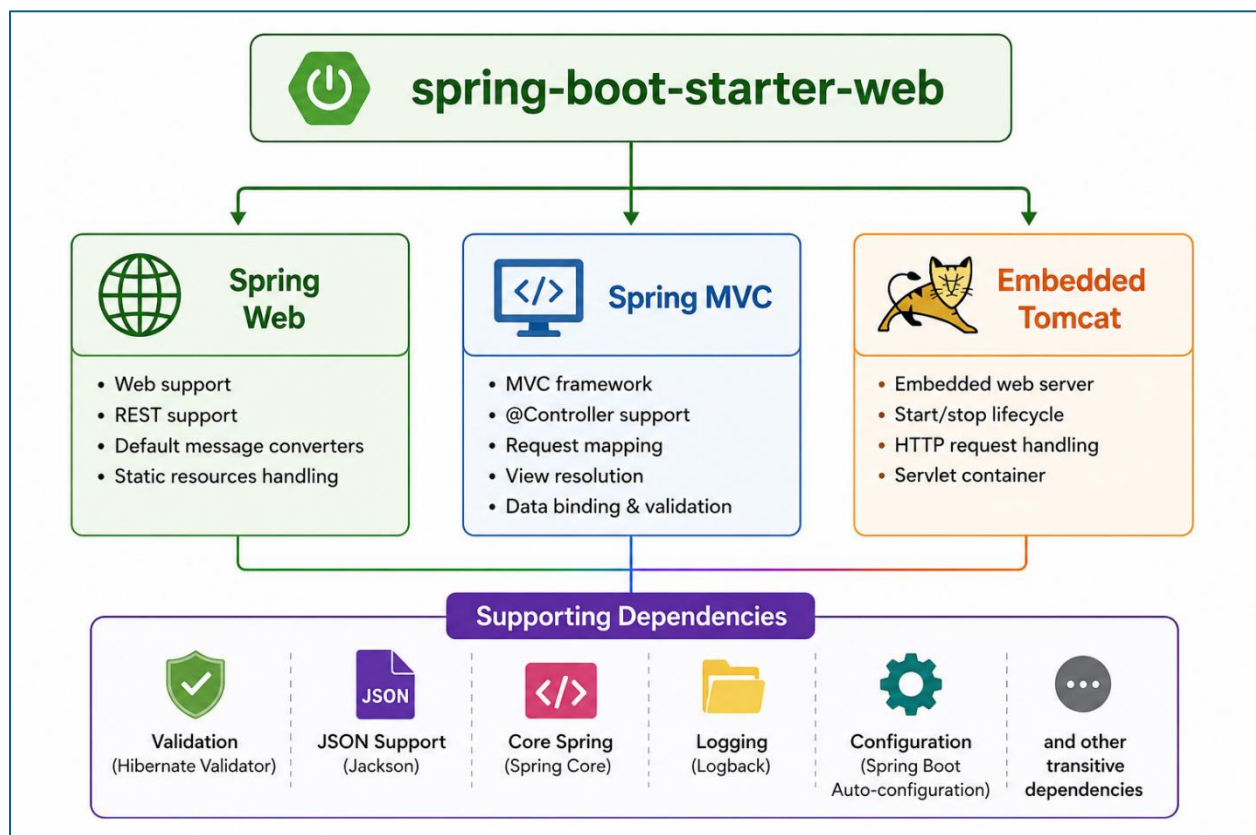
Instead of selecting numerous individual libraries, developers select the starter appropriate for the application's capability.

For example:

```
spring-boot-starter-web
```

- Spring Web
- Spring MVC
- JSON support
- validation-related support
- embedded Tomcat support
- other required dependencies

This reduces configuration and helps provide compatible dependency combinations.



5. Common Spring Boot Starters

Spring Boot provides several commonly used **starters**, each designed to support a specific area of application development.

Starter	Purpose
spring-boot-starter-web	Web applications and REST
spring-boot-starter-security	Security
spring-boot-starter-data-jpa	JPA/Hibernate data access
spring-boot-starter-thymeleaf	Thymeleaf templates
spring-boot-starter-actuator	Application monitoring and management

When these dependencies are selected in **Spring Initializr**, the corresponding entries are automatically added to `pom.xml`.

Tip

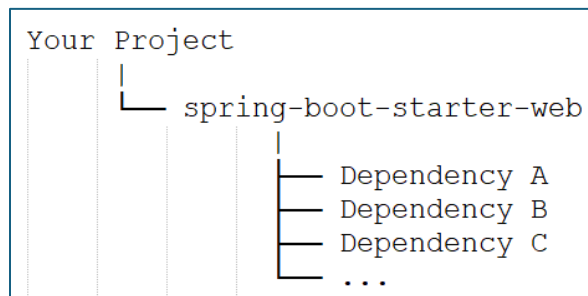
Think in terms of **application capabilities** rather than individual JAR files. If you need web support, start by asking which Spring Boot starter provides web support.

6. Inspecting Starter Dependencies

A starter may introduce many dependencies, including **transitive dependencies**.

The dependency hierarchy can be inspected using the Maven tools provided by your IDE. In **Eclipse**, you can use the **Dependency Hierarchy** view, while in **IntelliJ IDEA**, you can use the **Maven project/dependency view** to explore the project's dependencies.

Conceptually:



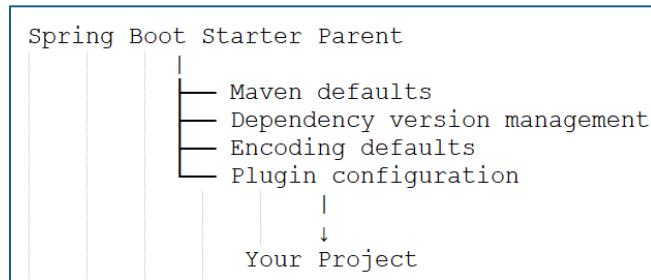
This is useful when troubleshooting dependency conflicts or determining which library introduced another library.

7. Spring Boot Starter Parent

Another important part of a Spring Boot Maven project is the **Spring Boot Starter Parent**.

The parent provides useful Maven defaults and dependency-management support.

Conceptually:



7.1 Dependency Version Management

One major advantage of the parent configuration is centralized dependency version management.

Instead of repeatedly specifying versions for managed dependencies, a project can rely on Spring Boot's compatible version set.

This improves:

Maintainability: Fewer individual dependency versions need to be maintained.

Compatibility: Spring Boot provides versions intended to work together.

Pitfall

Do not assume that a missing `<version>` in a Spring Boot dependency declaration means Maven does not know which version to use. The version may be supplied through Spring Boot's dependency management.

8. Configuring the Java Version

The Java version can be specified in the POM using a property such as:

```
<properties>
    <java.version>17</java.version>
</properties>
```


The exact version should match the Java environment required for the project.

9. Spring Boot Maven Plugin

A generated Spring Boot project can also include the:

```
spring-boot-maven-plugin
```

This plugin supports Spring Boot-specific build operations, including creating executable application archives and running the application.

For example:

```
./mvnw spring-boot:run
```

or, when Maven is installed:

```
mvn spring-boot:run
```

We will use this again when running applications from the command line.

10. Spring MVC Architecture

Spring MVC implements the **Model-View-Controller** architectural pattern for web applications.

Model

The **Model** represents the application's **data and business rules**.

In Spring applications, data can be represented using **POJOs (Plain Old Java Objects)**, while business logic is commonly implemented in **service components**.

View

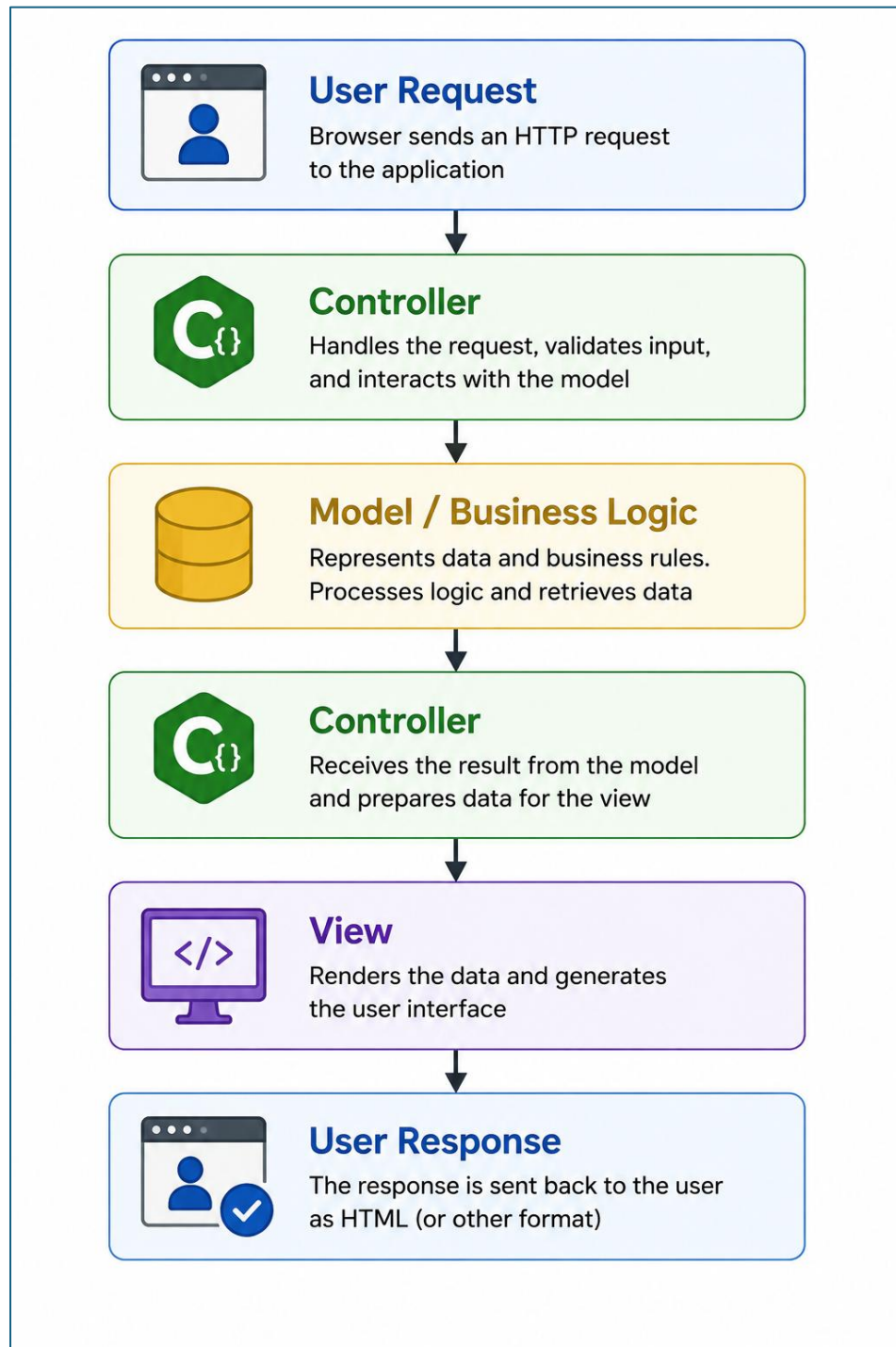
The **View** is responsible for **presenting information to the user**. It uses data provided by the Model to generate the user interface.

Common view technologies include:

- **JSP**
- **Thymeleaf**
- **FreeMarker**

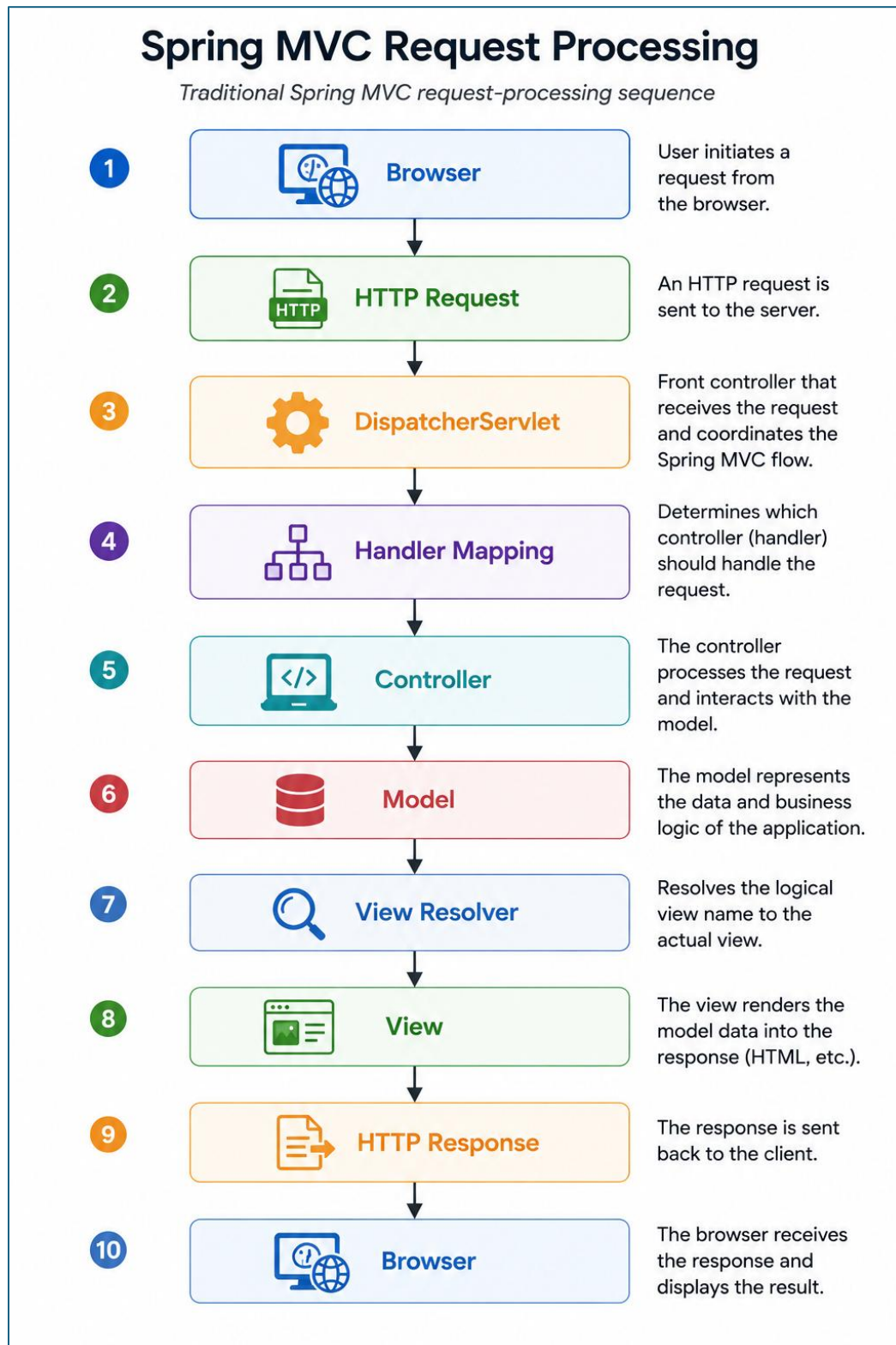
Controller

The **Controller** handles incoming HTTP requests and coordinates the application's response. It processes user input, interacts with the Model and application logic, and determines the appropriate response or View.



11. Spring MVC Request Processing

Spring MVC describes the *traditional* Spring MVC request-processing sequence as follows:



Step 1 — DispatcherServlet

The DispatcherServlet acts as Spring MVC's **front controller**.

Incoming requests first enter the Spring MVC processing infrastructure through it.

Step 2 — Handler Mapping

Spring determines which controller or handler should process the request.

Step 3 — Controller Execution

The selected controller executes and may interact with application business logic and model data.

Step 4 — View Resolution

For MVC applications returning views, a logical view name is mapped to an actual view.

Step 5 — Rendering

The selected view renders the model data.

Step 6 — Response

The generated response is returned to the client.

12. Spring Boot DevTools

During development, repeatedly stopping and restarting an application after every code modification slows the feedback cycle.

Spring Boot DevTools provides development-time functionality including automatic restart when relevant application files change.

13. Lab: Add Spring Boot DevTools

Step 1 — Add DevTools

Open `pom.xml` and add the Spring Boot DevTools dependency.

After Maven reloads the POM, the dependency becomes available to the application.

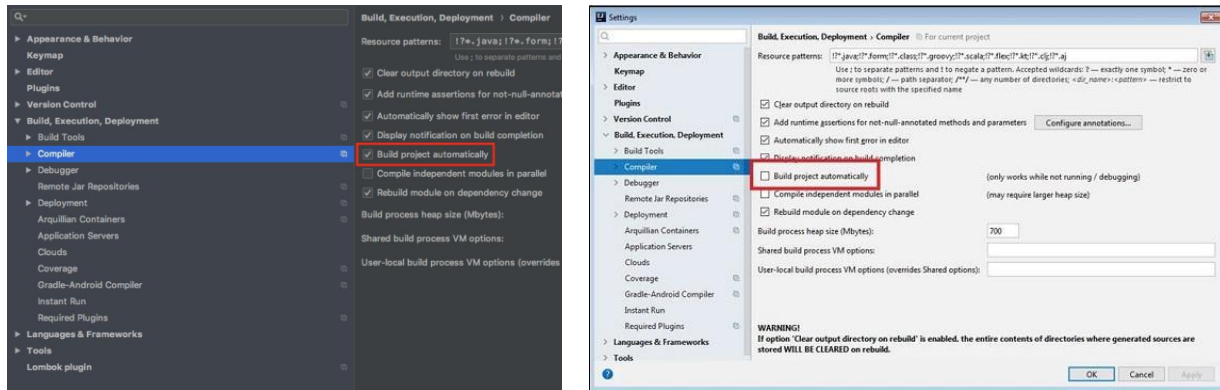
Step 2 — Configure IDE Build Behavior

a) Enable Build project automatically

Path: File → Settings → Build, Execution, Deployment → Compiler

Enable: ☒ Build project automatically

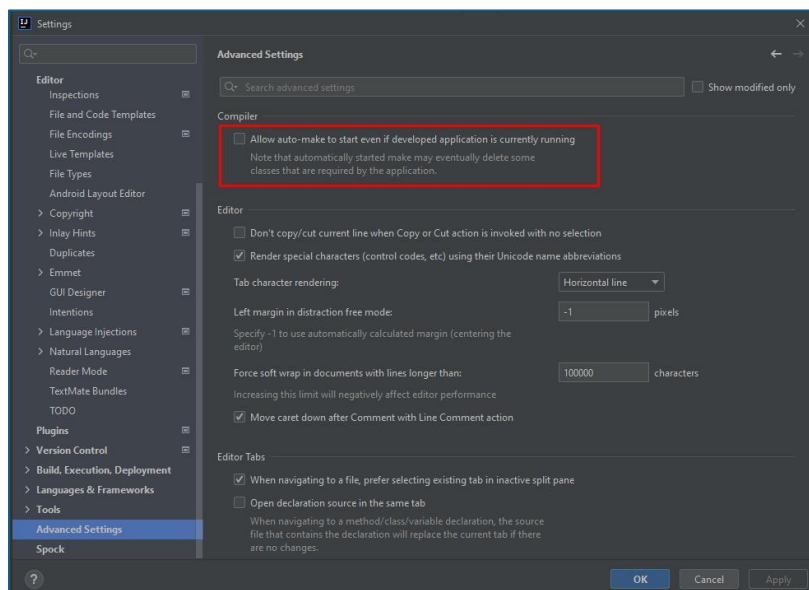
This tells IntelliJ IDEA to compile the project automatically when project files change.



b) Allow auto-make while the application is running

Path: File → Settings → Advanced Settings → Compiler

Enable: ☒ Allow auto-make to start even if developed application is currently running



This is particularly relevant when using **Spring Boot DevTools**, because it allows IntelliJ to rebuild changed classes while the Spring Boot application is running. JetBrains specifically recommends this together with **Build project automatically** for automatic reloading of changed Java classes.

Step 3 — Run the Application

Start the Spring Boot application.

Confirm that the embedded server starts.

Step 4 — Add a New Endpoint

For example:

```
@GetMapping("/workout")
public String getDailyWorkout() {
    return "Run a hard 5k!";
}
```

Step 5 — Save the File

After saving the source code, DevTools can trigger an application restart.

Watch the console for the restart activity.

Step 6 — Test

Navigate to:

`http://localhost:8080/workout`

Expected response:

Run a hard 5k!

Tip

DevTools is primarily a **development convenience**. Its purpose here is to shorten the edit → build → restart → test feedback loop.

14. Spring Boot Actuator

Once an application is running, developers and operations teams need information about its state.

Typical questions include:

- Is the application running correctly?

- Is it healthy?
- Which request mappings exist?
- Which Spring beans are loaded?
- What application information is available?

Spring Boot Actuator provides monitoring and management endpoints for this purpose.

15. Lab: Add Spring Boot Actuator

Step 1 — Add the Dependency

Add the Actuator starter to `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

The Actuator endpoints as being placed under `/actuator`

16. Important Actuator Endpoints

Examples include:

```
/actuator/health
/actuator/info
/actuator/beans
/actuator/mappings
/actuator/threaddump
```

These provide different views of application state and configuration.

/actuator/health

Provides health information. For example, it can indicate whether the application is operational.

/actuator/info

Can expose custom descriptive information about the application.

/actuator/beans

Provides information about Spring-managed beans.

/actuator/mappings

Shows request mappings.

/actuator/threaddump

Provides information about running threads.

17. Configuring Actuator Exposure

For example, use the following configuration in `application.properties` to expose the **health** and **info** Actuator endpoints:

```
management.endpoints.web.exposure.include=health,info
management.info.env.enabled=true
```

You can then test:

```
http://localhost:8080/actuator/health
```

and:

```
http://localhost:8080/actuator/info
```

18. Adding Custom Application Information

Add properties such as:

```
info.app.name=My Super Cool App
info.app.description=Description here
info.app.version=1.0.0
```

These values can then be made available through:

```
/actuator/info
```

19. Exposing Additional Actuator Endpoints

You can expose all endpoints using:

```
management.endpoints.web.exposure.include=*
```

You can then experiment with endpoints such as:


```
/actuator/beans  
/actuator/threaddump  
/actuator/mappings
```

20. Securing Actuator Endpoints

The next step is adding Spring Security.

Add:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

Adding this dependency causes protected endpoints to require authentication.

21. Default Credentials

By default, Spring Security uses:

Username: user

A generated password is displayed in the application console when the application starts.

You can override these default credentials in `application.properties`:

```
spring.security.user.name=scott  
spring.security.user.password=tiger
```

Warning

These simple credentials are suitable for learning and demonstration purposes. Production applications typically require a more robust security configuration, including proper authentication and authorization, securely encoded passwords, user roles and permissions, and potentially database-backed user management.

22. Excluding Actuator Endpoints

You can exclude specific Actuator endpoints from web exposure using the `management.endpoints.web.exposure.exclude` property.

For example, to exclude the **health** and **info** endpoints, add the following to `application.properties`:

```
management.endpoints.web.exposure.exclude=health,info
```

Once excluded, these endpoints will no longer be accessible over HTTP.

Practical Note

The health and info endpoints are often useful for **monitoring, DevOps, and application management**. Although they can be excluded when necessary, consider their operational value before disabling them.

23. Running Spring Boot from the Command Line

Spring Boot applications can be run directly from the **command line**, without launching the application from an IDE. This is useful for testing packaged applications and running them in environments where an IDE is not available.

There are two common approaches:

Option 1 — Run an Executable JAR

First, package the application as an executable JAR, and then run it using Java:

```
java -jar <application.jar>
```

For example:

```
java -jar target/mycoolapp-0.0.1-SNAPSHOT.jar
```

Because Spring Boot applications can include an **embedded web server**, the application and server start together.

Option 2 — Run with the Spring Boot Maven Plugin

You can also launch the application directly from the project directory using the Spring Boot Maven Plugin.

Windows:

```
mvnw spring-boot:run
```

macOS/Linux:

```
./mvnw spring-boot:run
```

If Maven is installed globally, you can use:

```
mvn spring-boot:run
```

Note

The key difference is that `java -jar` runs an **already packaged JAR**, while `spring-boot:run` launches the application through **Maven** from the project itself.

24. Lab: Build and Run an Executable Spring Boot JAR

Let's package the Spring Boot application and run it independently of the IDE.

Step 1 — Open a Terminal

Open a terminal or command prompt and navigate to the **root directory of the Spring Boot project**.

This is the directory containing:

```
pom.xml  
mvnw  
mvnw.cmd  
src/
```

For example:

```
cd mycoolapp
```

Step 2 — Package the Application

Run the Maven package goal.

Windows:

```
mvnw package
```

macOS/Linux:

```
./mvnw package
```

If Maven is installed globally, you can instead run:

```
mvn package
```

Maven will compile the application, run the configured tests, and package the application.

A successful build should end with a message similar to:

```
BUILD SUCCESS
```

Tip

If the build fails, review the Maven output before continuing. The JAR will not be generated successfully if the build does not complete.

Step 3 — Locate the Generated JAR

Maven places generated build artifacts in the:

```
target/
```

directory.

For example:

```
mycoolapp/  
├── pom.xml  
├── src/  
└── target/  
    └── mycoolapp-0.0.1-SNAPSHOT.jar
```

The exact JAR filename depends on the project's `artifactId` and `version`.

Step 4 — Run the JAR

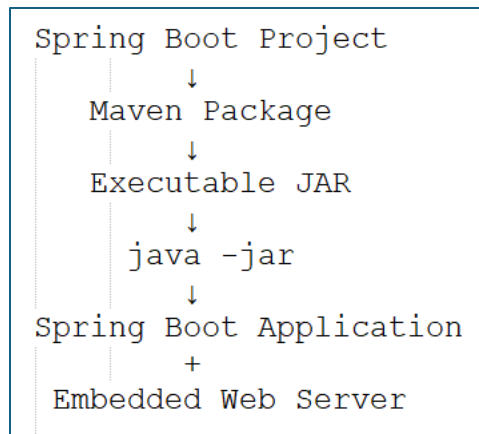
From the project root, execute:

```
java -jar target/mycoolapp-0.0.1-SNAPSHOT.jar
```

Replace the filename with the actual JAR generated for your project.

Spring Boot will start the application along with its **embedded web server**.

Conceptually:



Step 5 — Test the Application

Once the server has started, open a browser and test one of your application's endpoints.

For example:

`http://localhost:8080/`

or:

`http://localhost:8080/workout`

If you previously changed the server port or context path in `application.properties`, use the corresponding URL.

Step 6 — Stop the Application

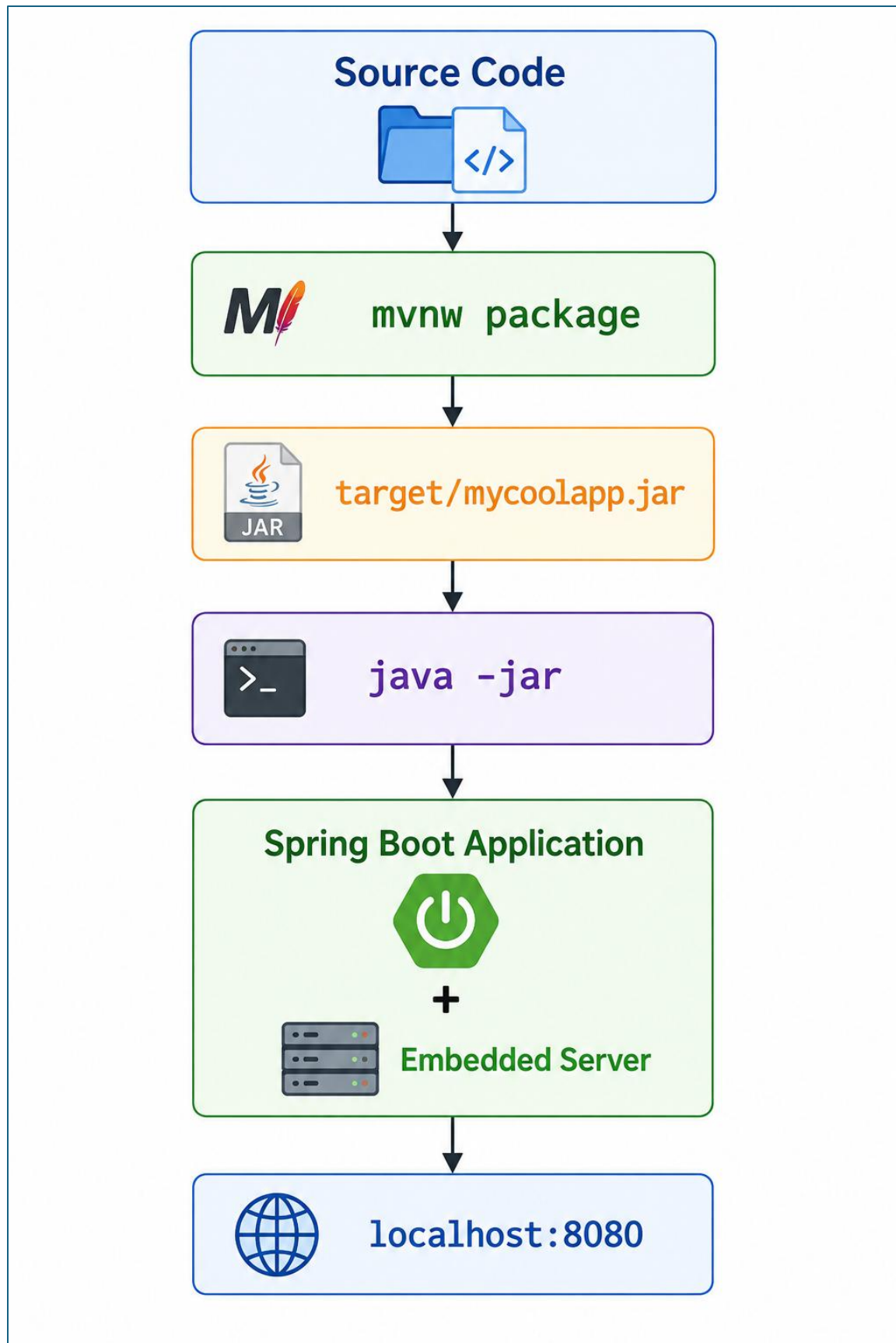
Return to the terminal and press:

`Ctrl + C`

This terminates the running Spring Boot application.

Practical Note

Running the executable JAR demonstrates an important advantage of Spring Boot: the application can operate as a **self-contained executable application** without requiring the IDE or a separately managed Tomcat installation.



25. Option 2 — Spring Boot Maven Plugin

Instead of packaging the application first, run:

macOS/Linux

```
./mvnw spring-boot:run
```

Windows

```
mvnw spring-boot:run
```

With Maven installed:

```
mvn spring-boot:run
```

The Spring Boot Maven plugin then launches the application.

26. Custom Application Properties

Hard-coding configurable values directly into Java classes makes applications harder to maintain.

Spring Boot uses:

```
src/main/resources/application.properties
```

as a standard location for application configuration.

For example:

```
coach.name=Mickey Mouse
```

```
team.name=The Mouse Club
```

27. Injecting Properties with @Value

Individual properties can be injected into a Spring-managed component using the `@Value` annotation. For example:

```
@Value("${coach.name}")
private String coachName;

@Value("${team.name}")
private String teamName;
```

The expression:

```
"${coach.name}"
```

instructs Spring to retrieve the value associated with the `coach.name` property from the application configuration and inject it into the `coachName` field.

Similarly:

```
"${team.name}"
```

retrieves the value of `team.name` and injects it into the `teamName` field.

28. Lab: Create a /teaminfo Endpoint

Given:

```
coach.name=Mickey Mouse
```

```
team.name=The Mouse Club
```

modify the controller:

```
@RestController
public class FunRestController {

    @Value("${coach.name}")
    private String coachName;

    @Value("${team.name}")
    private String teamName;

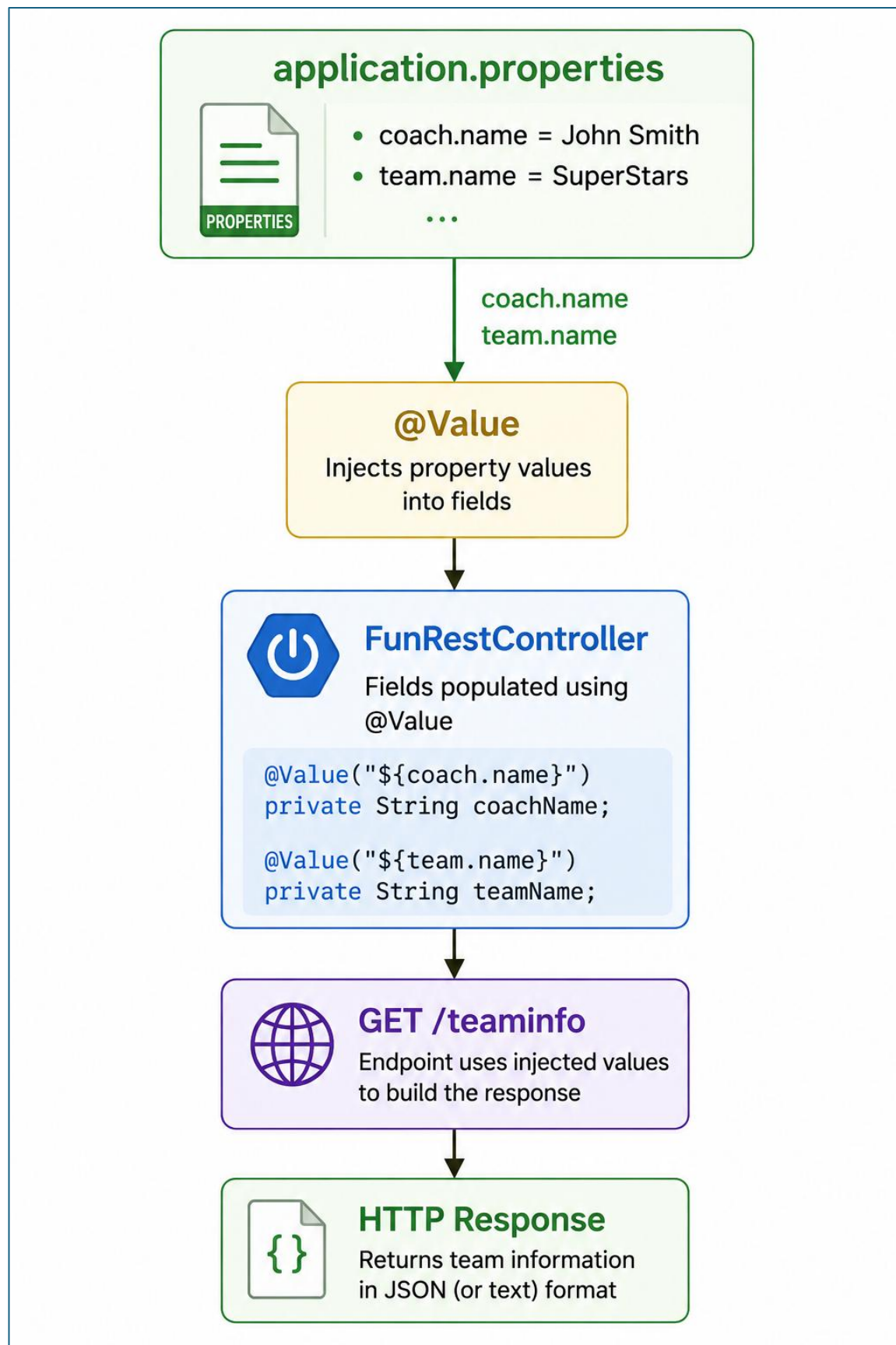
    @GetMapping("/teaminfo")
    public String getTeamInfo() {
        return "Coach: " + coachName
            + ", Team: " + teamName;
    }
}
```

Run the application and visit:

```
http://localhost:8080/teaminfo
```

The endpoint retrieves its values from `application.properties` instead of hard-coding them directly in the method. This process involves two main steps:

1. **Define the properties** in `application.properties`.
2. **Inject the property values** into the Java class using the `@Value` annotation.



29. Configuring the Embedded Server

`application.properties` can also be used to configure various aspects of the Spring Boot application and its infrastructure.

Common configuration categories include:

- **Core**
- **Web**
- **Security**
- **Data**
- **Actuator**
- **DevTools**
- **Testing**

For this lesson, we will focus specifically on **server configuration**, including settings such as the server port and application context path.

30. Lab: Change the Server Port

Spring Boot commonly runs the embedded server on port:

8080

To change it to 7070, add:

```
server.port=7070
```

Restart the application and inspect the console.

Then access:

```
http://localhost:7070/
```

instead of:

```
http://localhost:8080/
```

This demonstrates that configuration can modify application infrastructure without changing Java source code.

31. Configuring the Context Path

The **context path** is the base URL path under which a Spring Boot application is exposed. It acts as a prefix for all application endpoints.

For example, to configure the context path as `/mycoolapp`, add the following to `application.properties`:

```
server.servlet.context-path=/mycoolapp
```

If the application is running on port 7070, the base URL becomes:

```
http://localhost:7070/mycoolapp
```

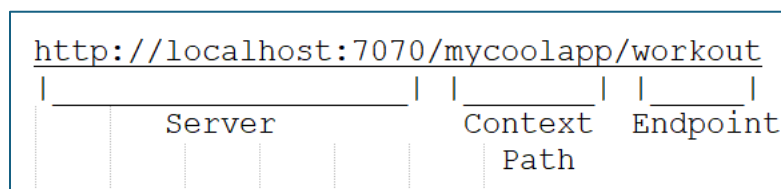
Suppose the application contains the following endpoint:

```
@GetMapping("/workout")
public String getDailyWorkout() {
    return "Run a hard 5k!";
}
```

The complete URL for this endpoint becomes:

```
http://localhost:7070/mycoolapp/workout
```

Conceptually:



The URL is therefore constructed as:

Server + Context Path + Endpoint

```
http://localhost:7070 + /mycoolapp + /workout
```

Pitfall

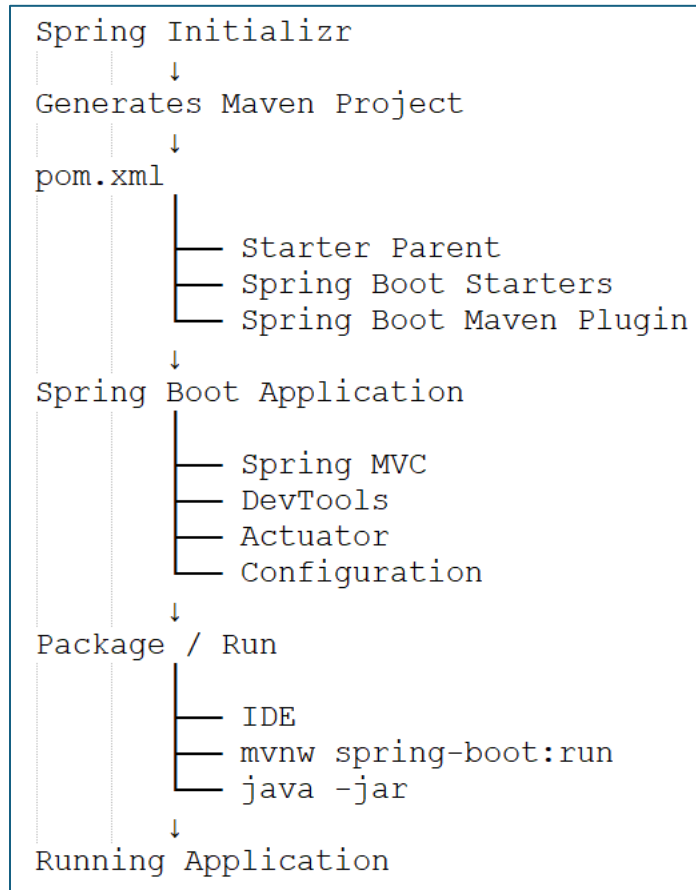
Changing the context path affects the URLs of **all application endpoints**. After configuring `/mycoolapp`, an endpoint such as `/workout` is no longer accessed directly as

```
http://localhost:7070/workout; the context path must be included:
```

```
http://localhost:7070/mycoolapp/workout.
```

32. Putting the Concepts Together

The concepts covered in this section form a connected **Spring Boot development workflow**:



This workflow shows how the major components work together—from **project creation and dependency management** to **application development, configuration, monitoring, packaging, and execution**.